Experiment No. – 7							
Date of Performance:	**1st Aug 2026**						
Date of Submission:	**1st Aug 2026**						
Conceptual Understanding (02)	**Execution (1.5)**	**Problem Solving & Debugging(02)**	**Professional Conduct & Lab Ethics(02)**	**Output/ Result Accuracy (1.5)**	**Documentation & Result Analysis (01)**	**Experiment Total (10)**	**Sign with Date**

Experiment No. 7
Implementation of Interfaces

7.1 Aim: Implementation of Interfaces : Create an interface vehicle and classes like bicycle, car, bike etc,having common functionalities and put all the common functionalities in the interface. Classes like Bicycle, Bike, car etc implement all these functionalities in their own class in their own way.

7.2 Course Outcome: Implement inheritance, interfaces, and packages to achieve code reusability and modular program design.

7.3 Learning Objectives: As java does not support multiple inheritance but there are occasions where a class needs to access data and methods from two or more classes in such cases Interfaces can be used and multiple inheritance can be achieved in java.

7.4 Requirement: Java development kit (JDK as per OS)

7.5 Related Theory:

An interface is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types. It defines a contract or a set of rules that classes implementing the interface must follow. An interface itself does not provide any implementation of

the methods declared within it; instead, it serves as a blueprint for classes to implement.
Declaration: An interface in Java is declared using the interface keyword followed by the interface name and a code block containing the method signatures and constant declarations.
Implementing Interfaces: A class can implement one or more interfaces by providing implementations for all the methods declared in those interfaces. This is done using the implements keyword in the class declaration.

Default Methods: In Java 8 and later versions, interfaces can have default methods, which are methods with a default implementation. Default methods allow adding new methods to interfaces without breaking existing implementations.
Static Methods: Java interfaces can also contain static methods, which are methods that belong to the interface itself rather than to any instance of the interface.
Multiple Inheritance: Unlike classes, a Java class can implement multiple interfaces, allowing for multiple inheritance of types.
Usage: Interfaces are commonly used to define contracts for classes that share common behavior but may have different implementations. They promote loose coupling and abstraction in Java programs.

An Interface in Java programming language is defined as an abstract type used to specify the behavior of a class. An interface in Java is a blueprint of a behavior. A Java interface contains static constants and abstract methods.
The interface in Java is *a* mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not the method body. It is used to achieve abstraction and multiple inheritances in Java using Interface. In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body. Java Interface also represents the IS-A relationship.When we decide on a type of entity by its behavior and not via attribute we should define it as an interface

Syntax for Java Interfaces

```
interface {
  // declare constant fields
  // declare methods that abstract
}
```

To declare an interface an interface keyword is used. It is used to provide total abstraction. That means all the methods in an interface are declared with an empty body and are public and all fields are public, static, and final by default. A class that implements an interface must implement all the methods declared in the interface. To implement the interface, use the implements keyword.

Uses of Interfaces in Java

It is used to achieve total abstraction.
Since java does not support multiple inheritances in the case of class, by using an interface it can achieve multiple inheritances.
Any class can extend only 1 class, but can any class implement an infinite number of interfaces.
It is also used to achieve loose coupling.
Interfaces are used to implement abstraction.

Advantages of Interfaces in Java
Without bothering about the implementation part, we can achieve the security of the implementation. In Java, multiple inheritance is not allowed, however, you can use an interface to make use of it as you can implement more than one interface.

Multiple Inheritance in Java Using Interface
Multiple Inheritance is an OOPs concept that can't be implemented in Java using classes. But we can use multiple inheritance in Java using Interface. let us check this with an example.

New Features Added in Interfaces in JDK 9
From Java 9 onwards, interfaces can contain the following also:

1. Static methods
2. Private methods
3. Private Static methods

Important Points in Java Interfaces
We can't create an instance (interface can't be instantiated) of the interface but we can make the reference of it that refers to the Object of its implementing class.
A class can implement more than one interface.
An interface can extend to another interface or interface (more than one interface).
A class that implements the interface must implement all the methods in the interface.
All the methods are public and abstract. And all the fields are public, static, and final.
It is used to achieve multiple inheritances.
It is used to achieve loose coupling.
Inside the Interface not possible to declare instance variables because by default variables are public static final.
Inside the Interface, constructors are not allowed.
Inside the interface main method is not allowed.
Inside the interface, static, final, and private methods declaration are not possible.

Table 7.1: Difference between Abstract class and Interface

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3) Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables.
4) Abstract class can provide the implementation of interface.	Interface can't provide the implementation of abstract class.
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
7) An abstract class can be extended using keyword "extends".	An interface can be implemented using keyword "implements".
8) A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.

9)Example:
public abstract class Shape{
public abstract void draw();
}
	Example:
	public interface Drawable{
	void draw();
	}

The relationship between classes and interfaces:

As shown in the figure given below, a class extends another class, an interface extends another interface, but a class implements an interface.

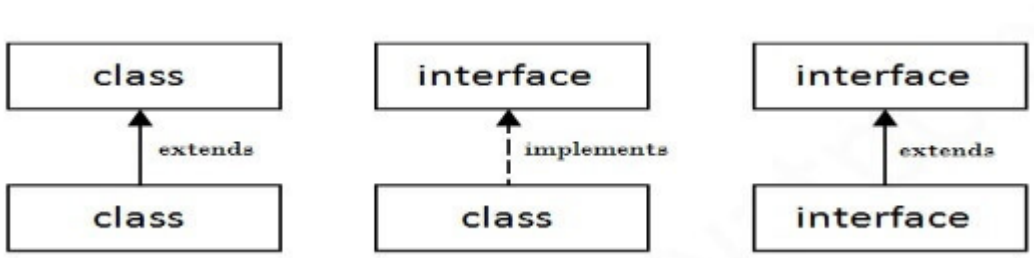


Figure 4.1: Relationship between classes and interfaces

Java Interface Example 1

The Printable interface has only one method, and its implementation is provided in the comp class.

```
interface printable
{
        void print();
}
class comp implements printable
{
                public void print()
                        {
                                System.out.println("Hello");
                        }

public static void main(String args[])
        {
                comp  obj = new comp ();
                obj.print();
         }
 }
```

Output: Hello

Java Interface Example 2

```
interface Bank
{
        float rateOfInterest();
}
class SBI implements Bank
{
        public float rateOfInterest(){return 9.15f;}
}
class PNB implements Bank
        {
                public float rateOfInterest()
                {
                        return 9.7f;
                }
```

```
        }
class RBI
{
public static void main(String[] args)
        {
                Bank b=new SBI();
                System.out.println("ROI: "+b.rateOfInterest());
        }
}
```

Output: ROI: 9.15

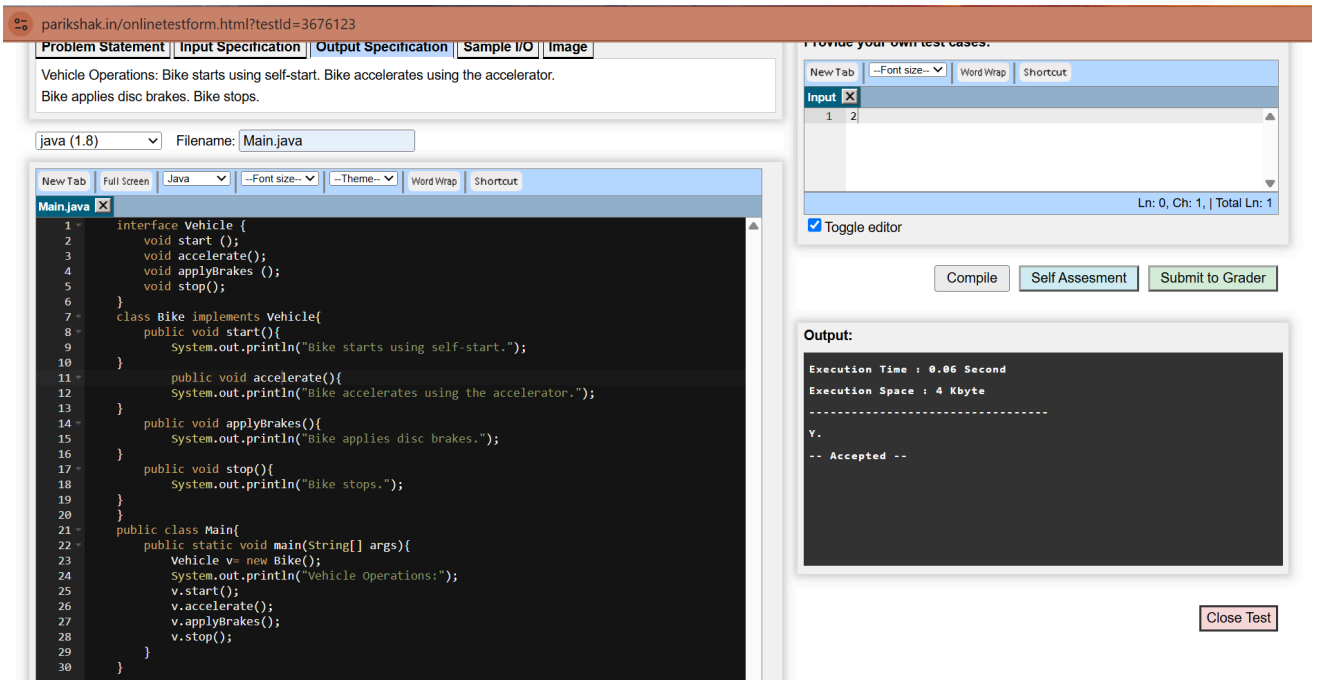
7.6 Procedure:

Create a interface with some abstract methods, create sub classes which will be implementing the abstract methods of interfaces as per their own requirement.

7.7 Program and Output:

```
interface Vehicle {
   void start();
   void accelerate();
   void applyBrakes();
   void stop();
}


class Bike implements Vehicle {


   public void start() {
      System.out.println("Bike starts using self-start.");
   }


   public void accelerate() {
      System.out.println("Bike accelerates using the accelerator.");
   }


   public void applyBrakes() {
      System.out.println("Bike applies disc brakes.");
   }
```

```
    public void stop() {
        System.out.println("Bike stops.");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle b = new Bike();

        System.out.println("Vehicle Operations:");
        b.start();
        b.accelerate();
        b.applyBrakes();
        b.stop();
    }
}
```



7.8 Conclusion:

This experiment demonstrated the implementation of interfaces in Java. We created a common Vehicle interface and implemented it in different classes such as Bicycle, Bike, and Car. Each class provided its own implementation of the interface methods, achieving abstraction and code reusability. Interfaces also help achieve multiple inheritance and promote loose coupling in Java applications.

7.9 Questions:

1. What are the components that can be included within an interface?

An interface can contain:

- Abstract methods
- Default methods
- Static methods
- Constants (public static final)
- Nested interfaces and classes

---

2. Can an interface have method implementations?

Yes. Since **Java 8**, interfaces can have **default** and **static** methods with implementations.

---

3. Can interfaces have fields or variables?

Yes. Interfaces can have only **public, static, and final** variables (constants).

---

4. How are static methods in interfaces different from default methods?

- **Static methods** belong to the interface and are called using the interface name.
- **Default methods** are inherited by implementing classes and can be overridden.

---

5. What is the purpose of static methods in interfaces?

Static methods provide utility functions related to the interface that can be called without creating an object.

---

6. Can an interface extend another interface?

Yes. One interface can extend one or more interfaces using the extends keyword.

Example:

```
interface A {
   void show();
}

interface B extends A {
   void display();
 }
```

---

7. Can an interface extend a class in Java?

No. An interface **cannot extend a class**. It can only extend another interface.

---

8. Can you provide an example scenario where interfaces are useful in Java programming?

A payment system can have a Payment interface implemented by classes such as CreditCard, DebitCard, and UPI. Each class processes payments differently while following the same interface, making the program flexible and reusable.